Linux Fest 2026

Infrastructure as Code

Why we stopped configuring servers by hand

By Mahdi Akbari



Where this hour goes

Six stops. Stop me anywhere .the demo is the part worth staying awake for.

1 **The cost of not having it**
What breaks when infrastructure lives only inside the machines.

2 **The core idea**
Declarative instead of imperative. Idempotent instead of hopeful.

3 **What can be code**
Four layers, from an empty VPC up to the workloads on top.

4 **How it gets applied**
Push or pull, and why the repo matters more than the tool.

5 **Writing it**
Markup, or a real language that generates the markup for you.

6 **The demo**
Two throwaway servers, one playbook, one act of vandalism.



What actually goes wrong without it

None of these show up as a line item. All of them show up as a bad week.

Environments drift apart

Staging passed, production did not. "Works on staging" stops being evidence of anything.

Onboarding takes weeks

The setup lives in one person's head and a folder of half-true notes.

Recovery is improvisation

There is no DR story, because nothing is reproducible on demand.

Changes are unreviewable

No diff, no approval, no audit trail. Just an SSH session nobody saw.

Scaling stays manual

The 31st server costs exactly as much effort as the 30th did.

Security quietly rots

A firewall rule somebody opened in 2023 that no one remembers or dares close.

Every one of these is the same root cause wearing a different costume.



The bill nobody puts on a ticket

The cost is real. It is just paid in places the roadmap does not track.

PEOPLE

- Knowledge lives in heads, not in a repo
- Bus factor of one on the boxes that matter

TIME

- Hours of manual setup per environment
- Nights spent reconstructing what changed

RISK

- Nobody can state the current state with confidence
- Recovery time you have never actually measured

None of it is laziness. It is what happens when the only copy of the truth is the running machine.



One question exposes all of it

If this server died right now, how long until an identical one is serving traffic?

With IaC

Minutes. A pipeline run. The definition is in Git, and the machine is disposable.

Without it

Hours, or days — and only if the person who originally built it is reachable.

The honest answer

"We would rebuild it. Roughly." Roughly is the word that should worry you.

Every problem on the last two slides collapses into one: the real state of your infrastructure exists only in the machine.



You already write Infrastructure as Code.

It is just not very good IaC yet.

- It tells the machine HOW, never WHAT
- It has no idea what the server looks like right now
- Run it twice and you get a different outcome
- It lives on somebody's laptop, not in Git

```
#!/bin/bash
# setup-web.sh

apt-get install -y nginx
mkdir /opt/app
useradd deploy
cp nginx.conf \
    /etc/nginx/nginx.conf
systemctl enable nginx
systemctl restart nginx
ufw allow 443

# this IS infra as code.
# just the fragile kind.
```



The runbook that works fine — once

01 Provision the VM

02 Create the deploy user

03 Install nginx

04 Copy the config file

05 Open port 443 in firewall

06 Request a TLS certificate

07 Mount the data disk

08 Install metrics agent

09 Set the timezone

10 Harden sshd

11 Reboot

12 Hope

30 servers



server #17 got step 7 wrong

How do you find out?

Honest answer: at 3 a.m., from a user.

Twelve steps, twelve chances to be slightly wrong, times thirty servers.

So what is Infrastructure as Code?

Infrastructure defined in machine-readable files, kept in version control and applied by a tool so that the files, not the running machines, are the source of truth.

Described, not performed

The file states what should be true, not the steps somebody happened to take.

→ next slide

Versioned

It lives in Git, so it has history, review, and a rollback that is just a revert.

→ the repo slide

Repeatable

Applying it twice is safe, and the second run changes nothing at all.

→ idempotency

Not this: a wiki page of commands, a GUI you clicked through, or a script that lives on one laptop.

`snowflake`
a hand-made server nobody dares touch

`drift`
identical servers slowly becoming different

`convergence`
the tool making reality match the file



HOW versus WHAT

IMPERATIVE

```
# a list of instructions
apt-get install -y nginx
systemctl enable nginx
systemctl start nginx
```

You describe the steps. If the steps do not match reality, that is your problem to handle.

DECLARATIVE

```
# a description of the end state
- name: nginx is running
  apt: { name: nginx,
         state: present }
  service: { name: nginx,
             state: started }
```

You describe the destination. The tool inspects reality and works out the steps itself.



Idempotency: run it twice

Safe to re-run, any number of times, with the same result.

The bash script

Re-runs blindly. mkdir on an existing directory fails, so the script dies halfway and leaves the server in a state nobody planned for.

```
mkdir: cannot create
  /opt/app: File exists
```

The playbook

Checks, then acts only if needed. Second run reports that everything already matches and changes nothing at all.

```
ok=6
changed=0
```

changed=0

The two most reassuring characters in this job.

This single property is why IaC scales to 30 servers, and why bash does not.



Four layers, four questions

Each layer answers a different question. Most teams end up needing more than one.

1	**Provisioning** Does the VM, network, disk or DNS record exist at all?	Terraform · OpenTofu · Pulumi
2	**Configuration** Is the right software on it, configured the right way?	Ansible · Chef · Puppet
3	**Images** Can I pre-build it so there is nothing to configure at boot?	Packer · Dockerfile
4	**Workloads** What actually runs on top of all of it?	Kubernetes · Helm · Argo CD

The next four slides take one layer each.





Provisioning: making it exist

Creates infrastructure that does not exist yet.

- VMs, VPCs, subnets, load balancers
- Managed databases, DNS zones, IAM roles
- Object storage buckets
- Talks straight to a cloud or hypervisor API

DEFINING TRAIT

It keeps state — a map from your declarations to real resource IDs. The API has no idea which of its ten thousand objects are yours.

```
# Terraform

resource "aws_instance" "web" {
  ami           = "ami-0abc123"
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.app.id

  tags = { Name = "web-01" }
}
```

Terraform · OpenTofu · Pulumi · CloudFormation · Bicep · Crossplane · Cluster API



Configuration: shaping the inside

Takes a machine that already exists and shapes its interior.

- Packages, config files, services, users
- Kernel parameters, certificates
- Pushed over SSH, or pulled by an agent
- It does not create servers

DEFINING TRAIT

Idempotent convergence: it makes an existing server match a described state. Ansible is stateless by design — it re-inspects the host every run instead of keeping a state file.

```
# Ansible

- name: nginx is installed and running
  hosts: web
  tasks:
    - apt:
        name: nginx
        state: present
    - service:
        name: nginx
        state: started
```

Ansible · Puppet · Chef · SaltStack · cloud-init



Images: configure once, boot ready

Produces an artifact that already contains the desired configuration.

- Nothing left to configure at boot
- Replaces configuration management
- The gateway to immutable infrastructure
- VM images, AMIs, OVAs, container images

DEFINING TRAIT

A Dockerfile is IaC. People forget that.

```
# Packer

source "amazon-ebs" "web" {
  source_ami    = "ami-0abc123"
  instance_type = "t3.micro"
}

build {
  sources = ["source.amazon-ebs.web"]
  provisioner "shell" {
    inline = ["apt-get install -y nginx"]
  }
}
```

Packer · Dockerfile + BuildKit · mkosi · debootstrap





Workloads: what runs on top

Defines what actually runs on the infrastructure.

- Kubernetes manifests, Helm charts
- Kustomize overlays, Argo CD Applications
- Nomad jobs, Docker Compose files
- The line with provisioning is blurry

DEFINING TRAIT

A Service of type LoadBalancer provisions a real cloud load balancer. Crossplane erases the line entirely, turning cloud resources into Kubernetes CRDs.

```
# Argo CD

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web
spec:
  source:
    repoURL: https://git/infra.git
    path: k8s/web
  destination:
    namespace: prod
  syncPolicy:
    automated: { selfHeal: true }
```

Kubernetes · Helm · Kustomize · Argo CD · Nomad · Compose





Push or pull: who starts the work?

The same split shows up twice in this field. It is worth being able to name it.

PUSH

Something outside the machine decides when to act.

- A human, or a CI job, runs the tool
- It connects over SSH or a cloud API
- Nothing is installed on the target
- It converges once, then stops watching

`Ansible · Terraform in a pipeline`

PULL

The machine asks what it should be, and fixes itself.

- An agent runs on the target, or in the cluster
- It fetches desired state from Git or a server
- It re-checks on a loop, forever
- Drift is corrected without anyone noticing

`Puppet · Chef · Argo CD · Flux`

Push tells you the state was correct when it ran. Pull tells you the state is correct now.



The tool matters less than the repo

IaC is really just applying the engineering habits you already learned to servers.

1

History

git log on your production configuration. Every change, dated, attributed.

2

Review

Infrastructure changes get read by a second human before they exist.

3

Rollback

A bad change is a revert, not an archaeology project.

4

Answers

“Who changed this, and why?” becomes a question with an answer.

A server you cannot rebuild from the repo is not managed. It is just running.



Two ways to write the same truth

CONFIG LANGUAGE

You write the state

Markup that IS the desired state. What you read in the file is what you get.

```
Terraform HCL
Ansible YAML
raw Kubernetes YAML
```

GENERAL-PURPOSE LANGUAGE

You write the generator

Real TypeScript or Python, with loops, functions, types and tests.

```
Pulumi
CDKTF (Terraform, in code)
cdk8s (Kubernetes, in code)
```



Your program does not deploy anything.

It builds a document. Something else deploys it.

This is called synthesis. Forget it and every other misconception follows.



The language is a generator, not a control loop. Nothing reaches your infrastructure until the engine at the far right runs.

Why you would want a real language

Copy, paste, pray

Forty lines of markup, three times over:

```
api-dev      ·  40 lines
api-staging  ·  40 lines                  replicas: 30
api-prod     ·  40 lines
```

Nobody reviews the third copy carefully.

One source, three outputs

```
for env in ["dev","staging","prod"]:
    make_service(
        name=f"api-{env}",
        replicas=3 if env=="prod"
                 else 1)
```

- Loops, functions, conditionals, types
- Editor autocomplete and real unit tests
- Share modules through pip or npm



Two servers, fifteen lines, five moves

Two throwaway VMs, one playbook, and one deliberate act of vandalism.

1	**Run it**	Both servers serve a templated page with their own hostname on it.
2	**Run it again**	changed=0. Nothing breaks. Idempotency, felt instead of explained.
3	**Break one by hand**	SSH in, mangle the config. This is configuration drift, live, on purpose.
4	**Re-run**	It heals itself back to the described state without being told what broke.
5	**Change one line**	Edit the template, re-run, both servers update together.

Terraform can then destroy both VMs and rebuild them identically from the same files.





Two videos worth your evening

VIDEO 01

What is Infrastructure as Code?

IBM Technology lightboard explainer.

youtu.be/zWw2wuiKd5o

VIDEO 02

Second recommended video

Picks up where this session stops.

youtu.be/POPP2WTJ8es

Three things to take away

1

Your infra is already code

The only question is whether it is good code: in Git, reviewed, and safe to run twice.

2

Idempotency is the game

Something you can re-run without fear is something you can automate, scale and recover.

3

Start with one layer

You do not need all four. Codify whatever hurts most and leave the rest for later.

Questions?

or come find me afterwards at telegram @mahdixak

